PROGRAMMING IN JAVA
ASSIGNMENT 3 – CO3

Student Name: M.Naga Shalini

Register Number:192524183

Q1 – Handling Failure in a Bank ATM Simulation

Aim

To develop an ATM withdrawal program that validates user input, checks the account balance, handles input and file-related

failures, and demonstrates try, catch, multi-catch and finally.

Exception hierarchy and classification

In Java, Throwable is the parent of Error and Exception. Errors generally indicate serious JVM or environment problems and

are not normally handled by application logic. Checked exceptions are checked by the compiler and must be caught or

declared. Unchecked exceptions are RuntimeException subclasses and usually indicate invalid input or programming

mistakes.

Failure analysis

1. Letters instead of an integer: InputMismatchException – unchecked; caught in the input catch block.
2. Negative or zero amount: IllegalArgumentException – unchecked; thrown by validation and caught.
3. Amount above balance: IllegalStateException – unchecked business-rule failure; caught separately.
4. Missing card-reader file: FileNotFoundException – checked; caught as IOException.

5. Insufficient memory or JVM failure: an Error; normally not caught as a routine transaction failure.

Java program

```
import java.util.*;
import java.io.*;
public class ATM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double balance = 5000.0;
        try {
            // Simulate checking the card-reader configuration file.
            File file = new File("cardreader.txt");
            if (!file.exists())
                throw new FileNotFoundException("cardreader.txt not found");
            System.out.print("Enter account number: ");
            int account = sc.nextInt();
            System.out.print("Enter withdrawal amount: ");
            int amount = sc.nextInt();
            if (amount <= 0)
                throw new IllegalArgumentException(
                    "Withdrawal amount must be positive");
            if (amount > balance)
                throw new IllegalStateException("Insufficient balance");
            balance = balance - amount;
            System.out.println("Account number: " + account);
            System.out.println("Cash dispensed: " + amount);
            System.out.println("Remaining balance: " + balance);
        } catch (InputMismatchException | IllegalArgumentException e) {
            System.out.println("Invalid input: " + e.getMessage());
        } catch (FileNotFoundException e) {
            System.out.println("Card reader error: " + e.getMessage());
        } catch (IllegalStateException e) {
```

```
            System.out.println("Transaction rejected: " + e.getMessage());
        } finally {
            System.out.println("ATM session ended.");
            sc.close();
        }
    }
}
```

Output:

Test1:Successful withdrawal

Enter account number: 101

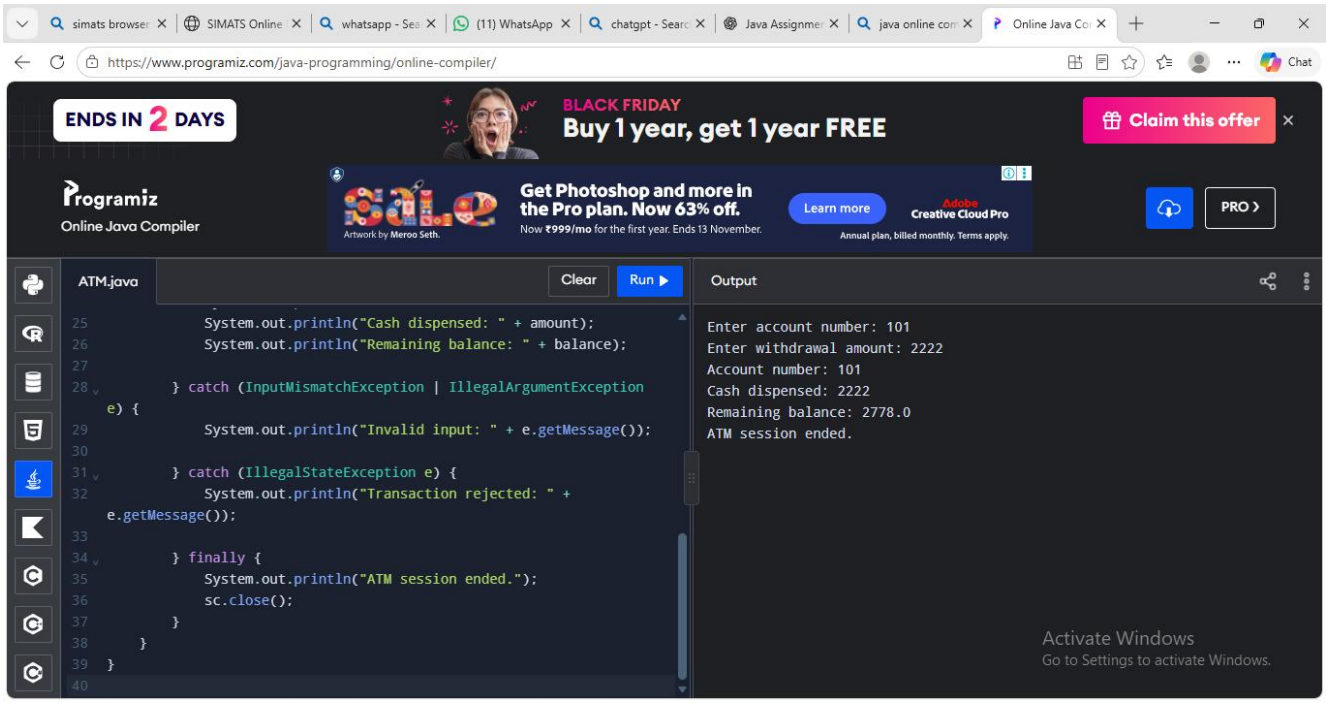
Enter withdrawal amount: 500

Account number: 101

Cash dispensed: 500

Remaining balance: 4500.0

ATM session ended.

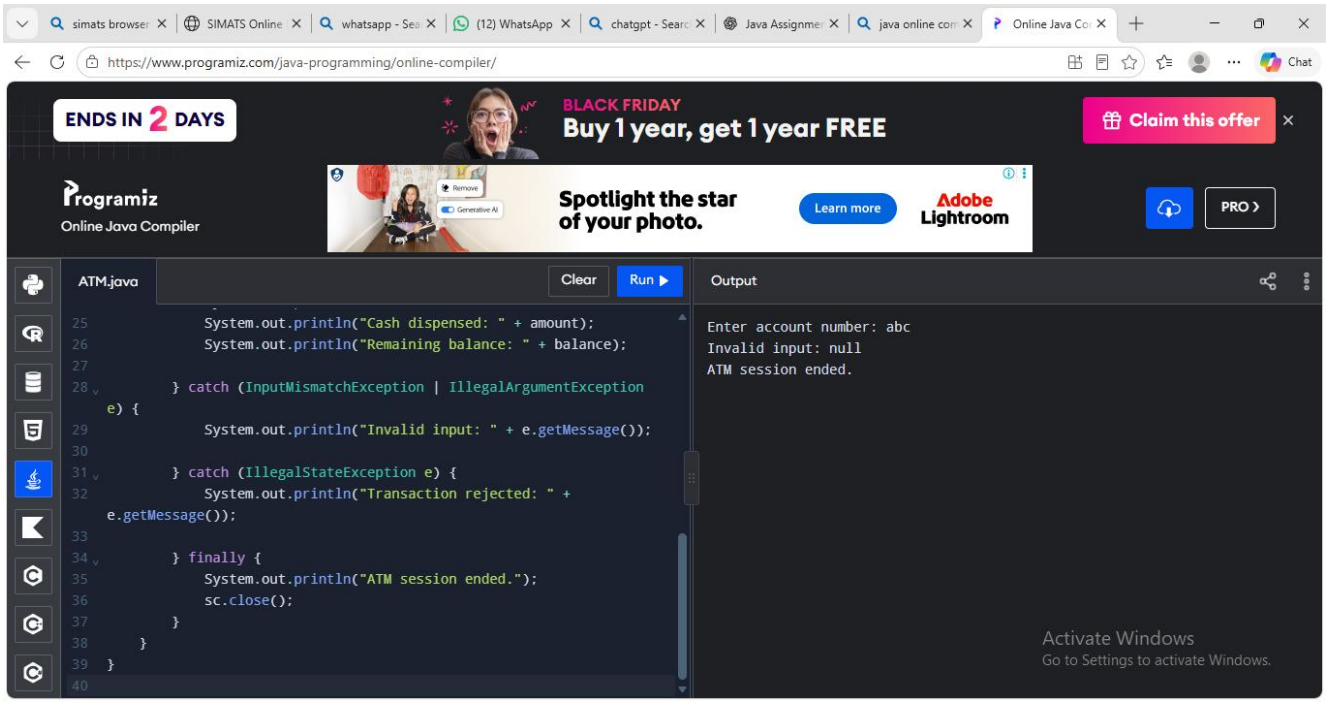


Test 2: Invalid account input

Enter account number: ABC

Invalid input: null

ATM session ended.



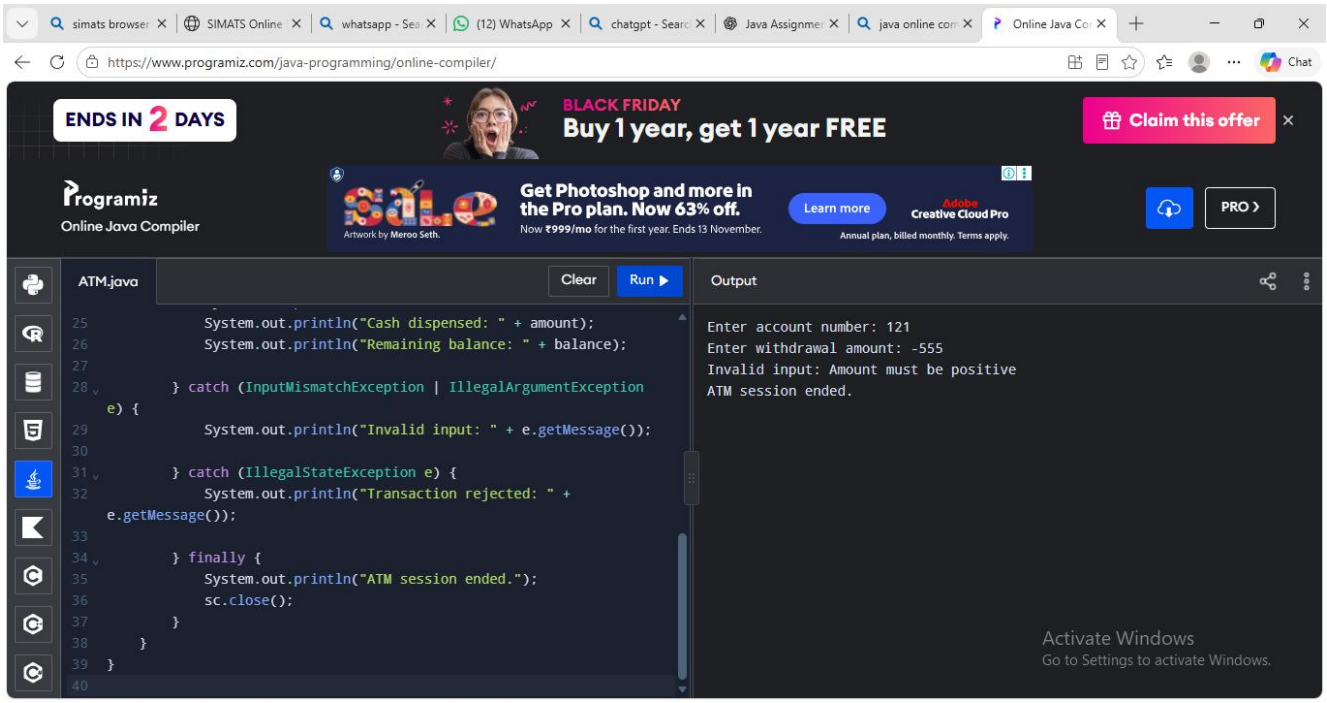
Test 3: Negative withdrawal

Enter account number: 101

Enter withdrawal amount: -50

Invalid input: Amount must be positive

ATM session ended.



Q2 – User-Defined Exceptions and Propagation in an Online

Aim and design

The portal rejects submissions when the exam window is closed, the student is not registered, or the answer file exceeds

the permitted size. The design uses a checked base exception, meaningful subclasses, throw/throws and exception
chaining.

Why checked exceptions?

These are expected business or service failures that the caller should consciously handle. Checked exceptions force the

controller or service to catch or declare them. Unchecked exceptions are more suitable for programming errors such as null

misuse or invalid internal state that should not be treated as normal portal outcomes.

Java program

import java.io.IOException;
class ExamException extends Exception {

```
    ExamException(String m) { super(m); }
    ExamException(String m, Throwable c) { super(m, c); }
}
class ExamWindowClosedException extends ExamException {
    ExamWindowClosedException(String m) { super(m); }
}
class StudentNotRegisteredException extends ExamException {
    StudentNotRegisteredException(String m) { super(m); }
}
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String m, Throwable c) {
        super(m, c);
    }
}
class Storage {
    void save(String fileName, long sizeKb)
            throws SubmissionStorageException {
        try {
            if (sizeKb > 500)
                throw new IOException("Disk quota exceeded");
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            throw new SubmissionStorageException(
                "Could not store " + fileName, e);
        }
    }
}
class ExamService {
    private final Storage storage = new Storage();
    void submit(boolean examOpen, boolean registered,
```

```
            String file, long size) throws ExamException {
        if (!examOpen)
            throw new ExamWindowClosedException("Exam window is
closed");
        if (!registered)
            throw new StudentNotRegisteredException(
                "Student is not registered for this course");
        storage.save(file, size);
    }
}
public class ExamPortalDemo {
    public static void main(String[] args) {
        ExamService service = new ExamService();
        try {
            service.submit(true, true, "answer01.pdf", 120);
            service.submit(true, true, "answer02.pdf", 800);
        } catch (ExamException e) {
            System.out.println("Error: " + e.getMessage());
            if (e.getCause() != null)
                System.out.println("Cause: " + e.getCause().getMessage());
        }
    }
}
```

Output

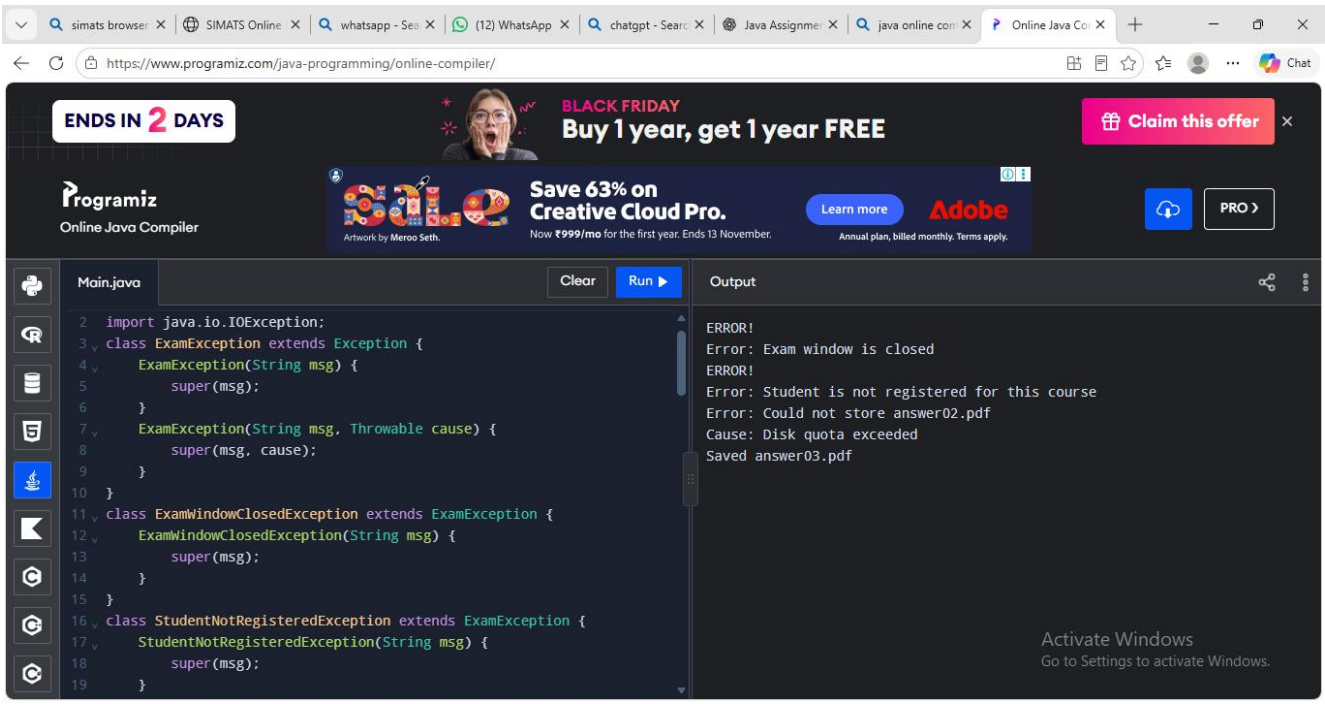
```
Error: Exam window is closed

Error: Student is not registered for this course

Error: Could not store answer02.pdf

Cause: Disk quota exceeded

saved
```



Q3 – Resource Safety, Debugging and Robust Code

Aim

Read multiple sales files and create a report while ensuring every opened resource is closed, even when a read or write

operation fails.

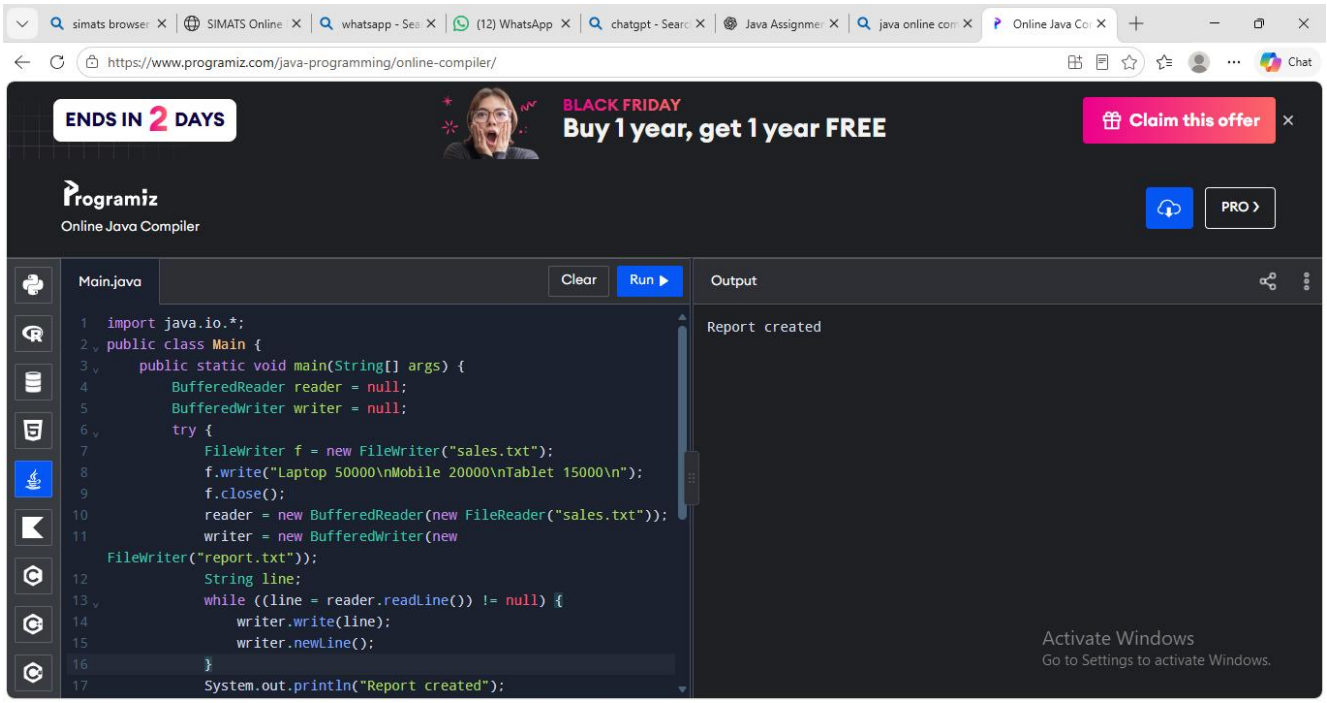
Java program

```
import java.io.*;
public class ManualReport {
   public static void main(String[] args) {
      BufferedReader reader = null;
      BufferedWriter writer = null;
      try {
         reader = new BufferedReader(new FileReader("sales.txt"));
         writer = new BufferedWriter(new FileWriter("report.txt"));
         String line;
         while ((line = reader.readLine()) != null) {
            writer.write(line);
```

```
            writer.newLine();
        }
        System.out.println("Report created");
    } catch (IOException e) {
        System.out.println("File error: " + e.getMessage());
    } finally {
        try {
            if (reader != null) reader.close();
        } catch (IOException e) {
            System.out.println("Reader close failed");
        }
        try {
            if (writer != null) writer.close();
        } catch (IOException e) {
            System.out.println("Writer close failed");
        }
    }
  }
}
```

Output:

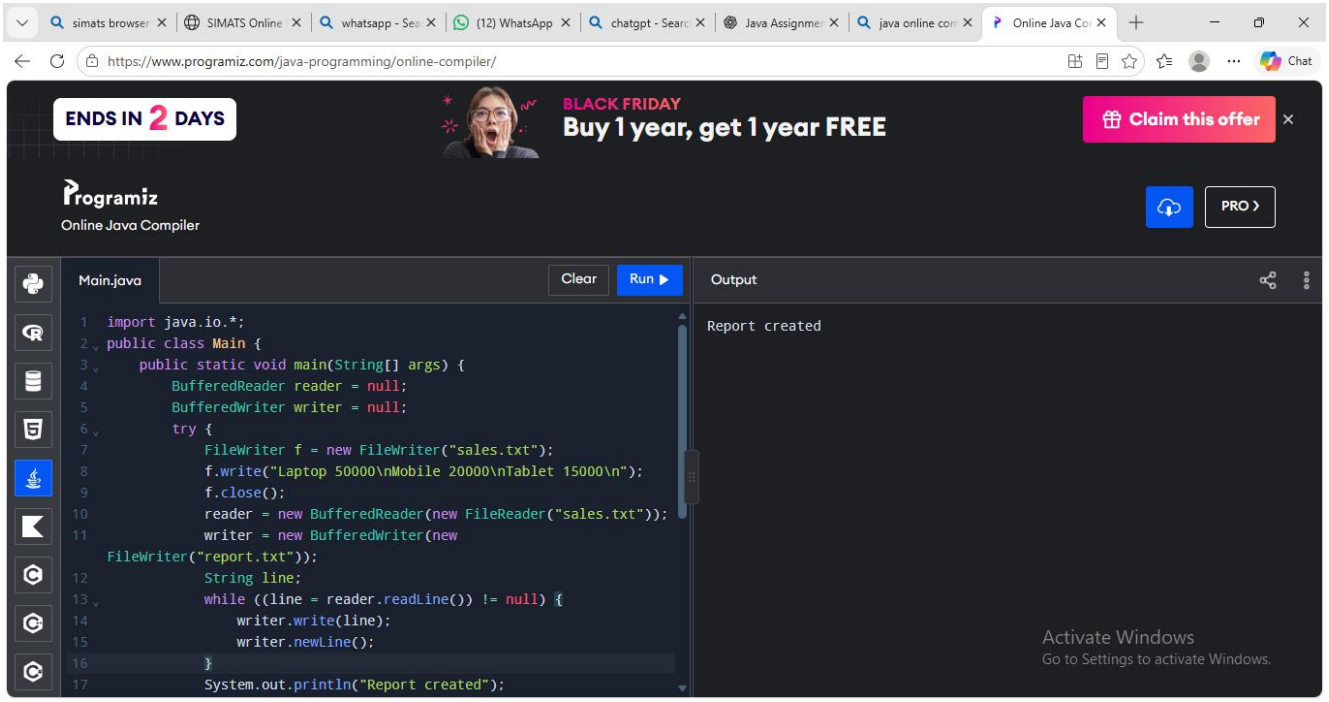
Report created



Resources implementation

```
import java.io.*;
public class SafeReport {
    public static void main(String[] args) {
        try (
            BufferedReader reader =
                new BufferedReader(new FileReader("sales.txt"));
            BufferedWriter writer =
                new BufferedWriter(new FileWriter("report.txt"))
        ) {
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }
            System.out.println("Report created");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
```

}




```
import java.io.*;
public class Main {
    public static void main(String[] args) {
        BufferedReader reader = null;
        BufferedWriter writer = null;
        try {
            FileWriter f = new FileWriter("sales.txt");
            f.write("Laptop 50000\nMobile 20000\nTablet 15000\n");
            f.close();
            reader = new BufferedReader(new FileReader("sales.txt"));
            writer = new BufferedWriter(new
FileWriter("report.txt"));
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }
            System.out.println("Report created");
```

Q4 – Thread Creation and the Thread Life Cycle

Aim

Simulate four scanner stations processing batches concurrently. A Java thread can be created by extending Thread or by

implementing Runnable. start() creates/schedules a separate execution path; run() called directly is just a normal method

call.

Version 1 – Extending Thread

```
class ScannerStation extends Thread {
    private String station;
    private int scripts;

    ScannerStation(String station, int scripts) {
        this.station = station;
```

```
        this.scripts = scripts;
    }

    public void run() {
        try {
            Thread.sleep(300);
            System.out.println(station + " scanned " + scripts);
        } catch (InterruptedException e) {
            System.out.println(station + " interrupted");
            Thread.currentThread().interrupt();
        }
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {

        ScannerStation[] stations = new ScannerStation[4];

        for (int i = 0; i < 4; i++) {
            stations[i] = new ScannerStation("Station " + (i + 1), 10);
            stations[i].start();
        }

        for (ScannerStation t : stations) {
            t.join();
```

```
        }

        System.out.println("All stations completed. Total = 40");
    }
}
```

Output:

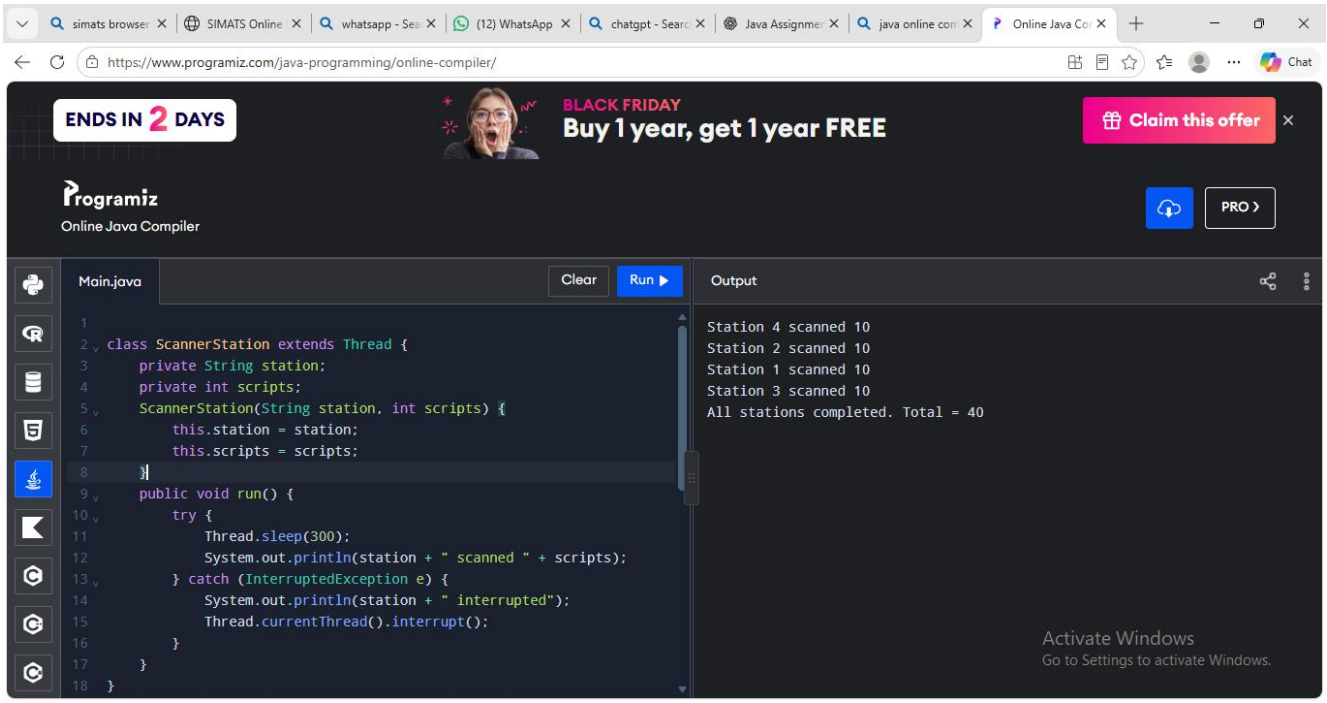
Station 1 scanned 10

Station 2 scanned 10

Station 3 scanned 10

Station 4 scanned 10

All stations completed. Total = 40



Version 2 – Implementing Runnable

```
class ScanTask implements Runnable {
    private String station;
    ScanTask(String station) {
        this.station = station;
```

```
    }
    public void run() {
        try {
            Thread.sleep(300);
            System.out.println(station + " completed scanning");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println(station + " interrupted");
        }
    }
}
public class Main {
    public static void main(String[] args)
            throws InterruptedException {
        Thread[] workers = new Thread[4];
        for (int i = 0; i < 4; i++) {
            workers[i] = new Thread(
                new ScanTask("Station " + (i + 1))
            );
            workers[i].start();
        }
        for (Thread t : workers) {
            t.join();
        }
        System.out.println("All stations completed.");
    }
}
```

Output:

Station 1 completed scanning

Station 2 completed scanning

Station 3 completed scanning

Station 4 completed scanning

All stations completed.

